

Loadable Kernel Module – Character device and timer functions

The objective of this lesson is to analyse, compile and test a Linux loadable kernel module (LKM) for a previously configured kernel.

The provided LKM periodically changes the state of a variable from all 1's to all 0's. This LKM can be used as a template for a blinker, with configurable blinking period, by adding the necessary code to change the state of the physical outputs.

Exercise I

1.1) Create a new directory for this exercise. Additionally, in that directory, create a new directory named blinker:

```
mkdir -p ~/lkm/blinker  
cd ~/lkm/blinker
```

1.2) Download the Makefile and module source code file, `blinker.c` to your working directory.

2) Analyse the code from `blinker.c` (Appendix A) and the documentation in Appendix B. For the time being, ignore the `module_param` and `MODULE_PARM_DESC` declarations.

- 2.1) Which function is executed when this module is loaded?
- 2.2) How do you show that this is a **character** device driver?
- 2.3) What is the major number for this device driver?
- 2.4) When is the `pisca_read` function called? How is that defined?
- 2.5) When is the `pisca_write` function called? How is that defined?
- 2.6) When is the `my_timer_func` function called? How is that defined?

3) Make the necessary changes to the `Makefile` file so that the LKM is built using the desired kernel configuration.

4) Compile the kernel module, by typing `make` at the command line in the LKM directory. You should obtain a new file named `blinker.ko` (the LKM). Unless you're using a Aarch64 computer, whenever you execute the `make` command, even for configuration, you must use the following command line parameters:

```
ARCH=arm64 CROSS_COMPILE=arm-
```

5) Copy the LKM to the Raspberry Pi.

6) On the Raspberry Pi:

6.1) Create a device file under `/dev` named `blinker` for the *blinker* device driver:

```
mknod /dev/blinker c major_number 0
```

where *major_number* is the major number used by the device driver. Inspect the source code to find this number.

6.2) Load the module using the `insmod` command. Confirm that the module is loaded, using `lsmod` or `cat /proc/modules`.

6.3) Check the value of the default blinking period. The device driver can be read using the following command:

```
cat /dev/blinker
```

6.4) Double the blinking frequency. Again, you should inspect the source code to find out which string should be written to `/dev/blinker` to obtain the desired blinking frequency.

You can send text strings to the device driver using the following command:

```
echo mystring > /dev/blinker
```

where *mystring* is the text to be written to `/dev/blinker`.

6.5) Check the kernel messages using the `dmesg` utility.

6.6) This module defines two variables as parameters: `led_status` and `blink_delay`. This is done through the following code:

```
module_param(led_status, byte, S_IRUSR|S_IWUSR);  
MODULE_PARM_DESC(led_status, "Port status");  
module_param(blink_delay, uint, S_IRUSR|S_IWUSR);  
MODULE_PARM_DESC(blink_delay, "Half period in jiffies");
```

Linux provides access to module parameters through the `sysfs` filesystem. This filesystem is usually mounted in `/sys`. In `/sys/module/` there is a directory for each module. Inside each module directory, there is a `parameters` directory, containing a file for each module parameter. In the present case, you should find the following two files:

```
/sys/module/blinker/parameters/blink_delay  
/sys/module/blinker/parameters/led_status
```

Note that `blink_delay` is the time the module waits before switching the value of `led_status` (and writing the new value to the physical address, see `my_timer_func`).

6.6.1) Set the period to 4 s by writing the appropriate value to `/sys/module/blinker/parameters/blink_delay`

6.6.2) Verify that the module is working as intended by running the following command repeatedly:

```
cat /sys/module/blinker/parameters/led_status
```

6.7) Shutdown the emulated machine by running `poweroff` or `halt` in the command line.

Exercise II

In what follows, the objective is to adapt the previous module so that it can be used to control the state of the GPIO 12 pin of the Raspberry Pi model 4 board.

1) Create a copy of the `blinker` directory and name it `blinker-rpi4`. All the following changes will be made in the files in the new directory:

```
cp -a blinker blinker-rpi4
cd blinker-rpi4
tar xvf linux-6.12-rpi4.tar.xz
```

2) Some changes must be made to the `Makefile`:

- Since the source code is contained in more than one C file, the `Makefile` must be changed accordingly. The kernel module will be named **`blinker-rpi4.ko`** and will be built from `blinker.o` and `gpio.o` (the list of files is specified through the **`blinker-rpi4-objs`** variable).
- In order to avoid the need of specifying the `ARCH` and `CROSS_COMPILE` parameters in the command line each time it is necessary to rebuild the module, these parameters are stored in the `Makefile` (`CCC_ARGS`) and passed to the `make` command by the `Makefile` itself.

Therefore, the `Makefile` should take the following form, where the `KERNEL_DIR` variable and `PATH_TO_ARM_TOOLCHAIN` should be adapted to the directory structure that you are using in your system:

```
obj-m +=blinker-rpi4.o
blinker-rpi4-objs := blinker.o gpio.o

KERNEL_DIR = ../linux-6.12-rpi4

#for non Aarch64 platforms
CCC_ARGS = ARCH=arm64 CROSS_COMPILE=arm-

all:
    make -C ${KERNEL_DIR} M=$(PWD) modules ${CCC_ARGS}
clean:
    make -C ${KERNEL_DIR} M=$(PWD) clean
```

3) Build the module.

4) Check the module's version with the `modinfo` command:

```
modinfo blinker-rpi4.ko
```

5) Test the module on the Raspberry Pi.

Exercise III

1) Implement a kernel module, **dimmer-rpi4.ko**, for the Raspberry Pi model 4 board, that produces a Pulse Width Modulation (PWM) signal, by software (“bit banging”), on the GPIO 12 pin. The module should be able to receive the duty cycle value through a device file. The duty cycle value should be an integer value between 0 (for always off) and 10 (for 100% of the time on).

In order to implement the PWM logic, consider a period of 10 jiffies and the duty cycle value as the number of jiffies to keep the GPIO pin active on each period. You can implement this logic in the timer callback function, as summarized below:

- If *new_period*
 - o *duty_cycle* = *duty_cycle_setpoint*
 - o If *duty_cycle* > 0
 - Set the GPIO PIN to 1
 - Set the timer for *duty_cycle* jiffies ahead
 - If *duty_cycle* < 10
 - Set *new_period* to 0 //Mid period transition from 1 to 0 occurs
//only when 0% < *duty_cycle* < 100%
 - o else
 - Set the GPIO PIN to 0
 - Set the timer for 10 jiffies ahead
- else
 - o Set the GPIO PIN to 0
 - o Set the timer for (10 – *duty_cycle*) jiffies ahead
 - o Set *new_period* to 1

2) Build the module and load it on the Raspberry Pi board.

3) Check the value of the HZ constant. What is the value of PWM frequency?

4) Test the module for different values of duty cycle.

5) Test the module on the Raspberry Pi.

Appendix A - Provided Files

Makefile:

```
obj-m := blinker.o

KDIR := linux kernel source code directory
PWD := $(shell pwd)

default:
    $(MAKE) -C $(KDIR) M=$(PWD) modules
```

blinker.c:

```
#include <linux/module.h>
#include <linux/fs.h>
#include <linux/cdev.h>
#include <linux/uaccess.h>

#define PISCA_MAJOR 500

MODULE_LICENSE("GPL");

#define RWBUFSIZE 11

static struct timer_list my_timer;
static unsigned char led_status = 0xFF;
static dev_t devno;
static struct cdev pisca_cdev;
static unsigned int blink_delay=3*HZ;
static int device_open = 0;

//check in /sys/module/blinker/parameters/
module_param(led_status, byte, S_IRUSR|S_IWUSR);
MODULE_PARM_DESC(led_status, "Port status");
module_param(blink_delay, uint, S_IRUSR|S_IWUSR);
MODULE_PARM_DESC(blink_delay, "Half period in jiffies");

static int pisca_open(struct inode *inode, struct file *filp)
{
    if (device_open)
    {
        printk(KERN_WARNING "Already open\n");
        return -EBUSY;
    }
    device_open++;
    try_module_get(THIS_MODULE);
    return 0;
}

int pisca_release(struct inode *inode, struct file *filp)
{
    device_open--;
    module_put(THIS_MODULE);
    return 0;
}

static ssize_t pisca_read(struct file *filp, char __user *buf, size_t count, loff_t *f_pos)
{
    static char local_buf[RWBUFSIZE];
    static int len;
    static unsigned int period_ms;
    int len1;
    int res;

    if((*f_pos)==0) {
        period_ms = blink_delay*1000/HZ*2;
    }
```

```

        sprintf(local_buf, "%d\n", period_ms);
        len = strlen(local_buf, RWBUFSIZE-1);
    }

    len1 = len - (*f_pos);
    len1 = len1 > count ? count : len1;

    res = copy_to_user(buf, local_buf + (*f_pos), len1);
    if(res!=0)
        printk(KERN_WARNING "Bytes left to copy\n");

    (*f_pos) += len1;

    return len1;
}

static ssize_t pisca_write(struct file *filp, const char __user *buf, size_t count, loff_t *f_pos)
{
    static char local_buf[RWBUFSIZE];
    int period_msec;
    int res, i;
    char c;

    for(i=0; i < count; ++i, ++(*f_pos))
    {
        if((*f_pos) > RWBUFSIZE - 2) //read \n and leave space for \0
            return -1;

        res = copy_from_user(&c, buf + i, 1);
        if(res!=0)
            printk(KERN_WARNING "Bytes left to copy\n");

        if(c == '\n')
        {
            local_buf[*f_pos] = 0;
            period_msec = simple_strtol(local_buf, NULL, 0);
            blink_delay = period_msec*HZ/2000;
            printk(KERN_WARNING "New period: %d ms\n", period_msec);
            return i+1;
        }
        else
            local_buf[*f_pos] = c;
    }

    return count;
}

static struct file_operations pisca_fops = {
    .owner =    THIS_MODULE,
    .read =    pisca_read,
    .write =    pisca_write,
    .open =    pisca_open,
    .release = pisca_release,
};

static void my_timer_func(struct timer_list *unused){
    led_status = ~led_status;
    //set the output to the new value
    //..

    my_timer.expires += blink_delay;
    add_timer(&my_timer);
}

int init_module(void){
    int result;

    devno = MKDEV(PISCA_MAJOR, 0);
    result = register_chrdev_region(devno, 1, "blinker");
    if (result < 0) {
        printk(KERN_WARNING "blinker: can't get major %d\n", PISCA_MAJOR);
        return result;
    }
}

```

```

cdev_init(&pisca_cdev, &pisca_fops);
pisca_cdev.owner = THIS_MODULE;
pisca_cdev.ops = &pisca_fops;
result = cdev_add (&pisca_cdev, devno, 1);
if (result)
    printk(KERN_NOTICE "Error %d", result);

printk(KERN_WARNING "HZ: %d\n", HZ);

//Initialization: set the output to a default value
//..

timer_setup(&my_timer, my_timer_func, 0);
my_timer.expires = jiffies + blink_delay;
add_timer(&my_timer);

return 0;
}

void cleanup_module(void){
    //shutdown: set the output to a default value
    //..

    del_timer(&my_timer);
    cdev_del(&pisca_cdev);
    unregister_chrdev_region(devno, 1);
}

```

Appendix B – API documentation

The documentation below consists of transcriptions from the following sources:

<https://www.kernel.org/doc/html/latest/driver-api/>

<https://www.kernel.org/doc/html/latest/core-api/>

<https://www.kernel.org/doc/html/latest/kernel-hacking/hacking.html#common-routines>

To check for a particular kernel version, replace “latest” by the desired version (e.g., **v6.12**) in the URL prefix. For instance:

<https://www.kernel.org/doc/html/v6.12/driver-api>

```
void add_timer(struct timer_list * timer)
```

start a timer

Parameters

```
struct timer_list * timer
```

the timer to be added

Description

The kernel will do a `->function(timer)` callback from the timer interrupt at the `->expires` point in the future. The current time is ‘jiffies’.

The timer’s `->expires`, `->function` fields must be set prior calling this function.

Timers with an `->expires` field in the past will be executed in the next timer tick.

```
int del_timer(struct timer_list * timer)
```

deactivate a timer.

Parameters

```
struct timer_list * timer
```

the timer to be deactivated

Description

[`del\_timer\(\)`](#) deactivates a timer - this works on both active and inactive timers.

The function returns whether it has deactivated a pending timer or not. (ie. [`del\_timer\(\)`](#) of an inactive timer returns 0, [`del\_timer\(\)`](#) of an active timer returns 1.)

```
int register_chrdev_region(dev_t from, unsigned count, const char * name)
```

register a range of device numbers

Parameters

dev_t *from*

the first in the desired range of device numbers; must include the major number.

unsigned *count*

the number of consecutive device numbers required

const char * *name*

the name of the device or driver.

Description

Return value is zero on success, a negative error code on failure.

```
void unregister_chrdev_region(dev_t from, unsigned count)
```

unregister a range of device numbers

Parameters

dev_t *from*

the first in the range of numbers to unregister

unsigned *count*

the number of device numbers to unregister

Description

This function will unregister a range of **count** device numbers, starting with **from**. The caller should normally be the one who allocated those numbers in the first place...

```
void cdev_init(struct cdev * cdev, const struct file_operations * fops)
```

initialize a cdev structure

Parameters

struct cdev * *cdev*

the structure to initialize

const struct file_operations * *fops*

the file_operations for this device

Description

Initializes **cdev**, remembering **fops**, making it ready to add to the system with [cdev_add\(\)](#).

```
int cdev_add(struct cdev * p, dev_t dev, unsigned count)
```

add a char device to the system

Parameters

```
struct cdev * p
```

the cdev structure for the device

```
dev_t dev
```

the first device number for which this device is responsible

```
unsigned count
```

the number of consecutive minor numbers corresponding to this device

Description

[`cdev\_add\(\)`](#) adds the device represented by **p** to the system, making it live immediately. A negative error code is returned on failure.

```
void cdev_del(struct cdev * p)
```

remove a cdev from the system

Parameters

```
struct cdev * p
```

the cdev structure to be removed

Description

[`cdev\_del\(\)`](#) removes **p** from the system, possibly freeing the structure itself.

NOTE

This guarantees that cdev device will no longer be able to be opened, however any cdevs already open will remain and their fops will still be callable even after `cdev_del` returns.

`try_module_get()` / `module_put()`

Defined in `include/linux/module.h`

These manipulate the module usage count, to protect against removal (a module also can't be removed if another module uses one of its exported symbols: see below). Before calling into module code, you should call `try_module_get()` on that module: if it fails, then the module is being removed and you should act as if it wasn't there. Otherwise, you can safely enter the module, and call `module_put()` when you're finished.

Most registerable structures have an owner field, such as in the `struct file_operations` structure. Set this field to the macro `THIS_MODULE`.

`copy_to_user()` / `copy_from_user()` / [`get\_user\(\)`](#) / [`put\_user\(\)`](#)

Defined in `include/linux/uaccess.h` / `asm/uaccess.h`

[`put\_user\(\)`](#) and [`get\_user\(\)`](#) are used to get and put single values (such as an int, char, or long) from and to userspace. A pointer into userspace should never be simply dereferenced: data should be copied using these routines. Both return `-EFAULT` or 0.

`copy_to_user()` and `copy_from_user()` are more general: they copy an arbitrary amount of data to and from userspace. Unlike [`put\_user\(\)`](#) and [`get\_user\(\)`](#), they return the amount of uncopied data (ie. 0 still means success).

Appendix C – Kernel log levels

From /usr/src/kernels/4.18.16-200.fc28.x86_64/include/linux/kern_levels.h:

```
#define KERN_SOH      "\001"          /* ASCII Start Of Header */
#define KERN_SOH_ASCII '\001'

#define KERN_EMERG     KERN_SOH "0"  /* system is unusable */
#define KERN_ALERT     KERN_SOH "1"  /* action must be taken immediately */
#define KERN_CRIT      KERN_SOH "2"  /* critical conditions */
#define KERN_ERR        KERN_SOH "3"  /* error conditions */
#define KERN_WARNING   KERN_SOH "4"  /* warning conditions */
#define KERN_NOTICE     KERN_SOH "5"  /* normal but significant condition */
#define KERN_INFO       KERN_SOH "6"  /* informational */
#define KERN_DEBUG      KERN_SOH "7"  /* debug-level messages */
```

From /usr/src/kernels/4.18.16-200.fc28.x86_64/include/linux/printk.h:

```
#define pr_emerg(fmt, ...) \
    printk(KERN_EMERG pr_fmt(fmt), ##__VA_ARGS__)
#define pr_alert(fmt, ...) \
    printk(KERN_ALERT pr_fmt(fmt), ##__VA_ARGS__)
#define pr_crit(fmt, ...) \
    printk(KERN_CRIT pr_fmt(fmt), ##__VA_ARGS__)
#define pr_err(fmt, ...) \
    printk(KERN_ERR pr_fmt(fmt), ##__VA_ARGS__)
#define pr_warning(fmt, ...) \
    printk(KERN_WARNING pr_fmt(fmt), ##__VA_ARGS__)
#define pr_warn pr_warning
#define pr_notice(fmt, ...) \
    printk(KERN_NOTICE pr_fmt(fmt), ##__VA_ARGS__)
#define pr_info(fmt, ...) \
    printk(KERN_INFO pr_fmt(fmt), ##__VA_ARGS__)
```